بسم الله الرحمن الرحيم

King Abdulaziz University
Faculty of Computing and information Technology
Computer Science Department

CPCS-214 Computer Architecture and Organization

11 - Signed and Unsigned Numbers

Introduction

- Humans are taught to think in base 10
- How a computer represents numbers?

Decimal and Binary

- Numbers may be represented in any base
- For example, 123 base 10 = 1111011 base 2
- Numbers are kept in computer HW as a series of high and low electronic signals
  - So they are considered base 2 numbers
- As base 10 numbers are called *decimal* numbers, base 2 numbers are called *binary* numbers

Binary Digits

- Single digit of binary number is “atom” of computing, since all information is composed of binary digits or *bits*
- This fundamental building block can be one of two values:
  - high or low
  - on or off
  - true or false
  - 1 or 0

Digit Representation

- Generalizing, in any number base, value of *i*th digit *d* is

$$d \times Base^{i}$$

where *i* starts at 0 and increases from RTL

- Leads to obvious way to number bits in word:
  - Use power of base for that bit

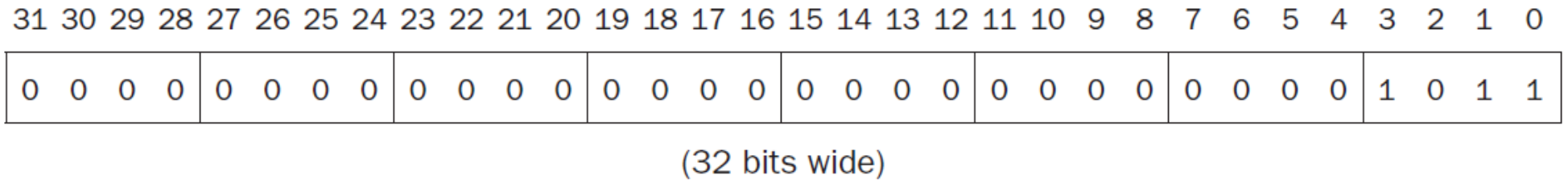
Binary Numbers

- Subscript decimal numbers with *ten*, binary numbers with *two*
- For example, 1011_{two} represents

$$(1 \times 2^3) + (0 \times 2^2) + (1 \times 2^1) + (1 \times 2^0)_{ten}$$
$$= (1 \times 8) + (0 \times 4) + (1 \times 2) + (1 \times 1)_{ten}$$
$$= 8 + 0 + 2 + 1_{ten}$$
$$= 11_{ten}$$

Binary Number Representation

- We number bits 0, 1, 2, 3, . . . from *RTL* in a word
- Below shows numbering of bits within a MIPS word and the placement of the number 1011_{two}:



LSB and MSB

- Since words are drawn vertically as well as horizontally, left most and rightmost may be unclear
- Hence, phrase
  - least significant bit (LSB) used to refer to rightmost bit (bit 0)
  - most significant bit (MSB) to leftmost bit (bit 31)

32-bit Binary Numbers

- MIPS word is 32 bits long
- So we can represent 2^{32} different 32-bit patterns
  - Combinations represent numbers: 0 to $2^{32} - 1$ ($4{,}294{,}967{,}295_{ten}$)

32-bit Binary Numbers

- MIPS word is 32 bits long
- So we can represent 2^{32} different 32-bit patterns
  - Combinations represent numbers: 0 to $2^{32} - 1$ ($4{,}294{,}967{,}295_{ten}$)

$0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000_{two^{two}} = 0_{ten}$

$0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0001_{two} = 1_{ten}$

$0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0010_{two} = 2_{ten}$

. . . . . .

$1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1101_{two} = 4{,}294{,}967{,}293_{ten}$

$1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1110_{two} = 4{,}294{,}967{,}294_{ten}$

$1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111_{two} = 4{,}294{,}967{,}295_{ten}$

Unsigned Numbers

- 32-bit binary numbers can be represented in terms of bit value times a power of 2 (here *xi* means *i*th bit of *x*):

$$(x31 \times 2^{31}) + (x30 \times 2^{30}) + (x29 \times 2^{29}) + \ldots + (x1 \times 2^{1}) + (x0 \times 2^{0})$$

- These positive numbers are called unsigned numbers

Overflow

- HW can be designed to add, subtract, multiply, and divide binary bit patterns
- If the number that is the proper result of such operations cannot be represented by rightmost HW bits, *overflow* is said to have occurred

Signed Numbers

- Computer programs calculate both positive and negative numbers
- So we need a representation that distinguishes positive from negative

Sign and Magnitude

- Most obvious solution is to add a separate sign
- Conveniently can be represented in a single bit
- Name for this representation is *sign and magnitude*
- Has several shortcomings

Sign and Magnitude Shortcomings

- Where to put sign bit; right? or left ?
  - Early computers tried both
- Adders may need an extra step to set sign
  - Can't know in advance what the proper sign will be
- Separate sign bit means to has both positive and negative zero
  - Can lead to problems for inattentive programmers
- As a result sign and magnitude representation abandoned

Alternative

- What would be the result for unsigned numbers if we tried to subtract a large number from a small one?
- Answer is that it would try to borrow from a string of leading 0s, so result would have a string of leading 1s
- Final solution is to pick representation that made HW simple
  - Leading 0s mean positive, and leading 1s mean negative
- *Two's complement* representation

Two's Complement Signed Numbers

- Convention for representing signed binary numbers is called *two's complement* representation

$$
\begin{aligned}
0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000_{two} &= 0_{ten} \\
0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0001_{two} &= 1_{ten} \\
0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0010_{two} &= 2_{ten} \\
\ldots & \ldots \\
0111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1101_{two} &= 2{,}147{,}483{,}645_{ten} \\
0111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1110_{two} &= 2{,}147{,}483{,}646_{ten} \\
0111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111_{two} &= 2{,}147{,}483{,}647_{ten} \\
1000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000_{two} &= -2{,}147{,}483{,}648_{ten} \\
1000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0001_{two} &= -2{,}147{,}483{,}647_{ten} \\
1000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0010_{two} &= -2{,}147{,}483{,}646_{ten} \\
\ldots & \ldots \\
1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1101_{two} &= -3_{ten} \\
1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1110_{two} &= -2_{ten} \\
1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111_{two} &= -1_{ten}
\end{aligned}
$$

Two's Complement Signed Numbers

- Positive half of numbers, from 0 to $2{,}147{,}483{,}647_{ten}$ ($2^{31} - 1$), use same representation as before
- Bit pattern ($1000\ldots0000_{two}$) represents the most negative number $2{,}147{,}483{,}648_{ten}$ (2^{31})
- Followed by a declining set of negative numbers: $2{,}147{,}483{,}647_{ten}$ ($1000\ldots0001_{two}$) down to 1_{ten} ($1111\ldots1111_{two}$)

Two's Complement Signed Numbers

- One negative number, $2{,}147{,}483{,}648_{ten}$, has no corresponding positive number
- Every computer today uses two's complement binary representations for signed numbers

Sign Bit

- Advantage that all negative numbers have a 1 in MSB
  - HW needs to test only this bit to see if a number is positive or negative (with number 0 considered positive)
- This bit is often called the *sign bit*

Two's Complement Format

- We can represent positive and negative 32-bit numbers in terms of the bit value times a power of 2:

$$(x31 \times -2^{31}) + (x30 \times 2^{30}) + (x29 \times 2^{29}) + \ldots + (x1 \times 2^{1}) + (x0 \times 2^{0})$$

- Sign bit is multiplied by -2^{31}, and the rest of the bits are then multiplied by positive versions of their respective base values

Binary to Decimal Conversion

- What is the decimal value of the 32-bit two's complement number?

$$1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1100_{two}$$

- Substituting the number's bit values into the formula:

$$\begin{aligned}
&(1 \times -2^{31}) + (1 \times 2^{30}) + (1 \times 2^{29}) + \ldots + (1 \times 2^{1}) + (0 \times 2^{1}) + (0 \times 2^{0}) \\
&= -2^{31} + 2^{30} + 2^{29} + \ldots + 2^{2} + 0 + 0 \\
&= -2{,}147{,}483{,}648_{ten} + 2{,}147{,}483{,}644_{ten} \\
&= -4_{ten}
\end{aligned}$$

Overflow

- Occurs when the left most retained bit of the binary bit pattern is not the same as the infinite number of digits to left (sign bit is incorrect)
  - 0 on left of bit pattern when number is negative or
  - 1 when number is positive

Sign Extension

- Signed versus unsigned applies to loads as well as to arithmetic
- Signed load is to copy sign repeatedly to fill rest of register
  - Called *sign extension*
  - To place a correct representation of the number within that register
- Unsigned loads fill with 0s to the left of data,
  - Since number represented by the bit pattern is unsigned

Load Byte

- When loading a 32-bit word into a 32-bit register, signed and unsigned loads are identical
- MIPS does offer two flavors of byte loads
  - *load byte* (`lb`) treats byte as a signed number and thus sign-extends to fill the 24 left-most bits of the register
  - *load byte unsigned* (`lbu`) works with unsigned integers
    - Since C programs use bytes to represent characters rather than consider bytes as very short signed integers, `lbu` is used exclusively for byte loads

Signed and Unsigned Numbers

- Unlike numbers, memory addresses naturally start at 0 and continue to the largest address
  - Negative addresses make no sense
- Programs want to deal sometimes with numbers that can be
  - Positive or negative
  - Sometimes with only positive numbers

Negation

- Quick way to negate a two's complement binary number
- Invert every 0 to 1 and every 1 to 0, then add one to result
- Based on observation that sum of number and its inverted representation must be $111 \ldots 111_{two}$, which represents -1
  - Since $x + x' = -1$, therefore $x + x' + 1 = 0$ or $x' + 1 = -x$

Example

- Negate 2_{ten}, and then check the result by negating -2_{ten}

Negating this number by inverting the bits and adding one,

$$\begin{array}{ll} & 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1101_{two} \\ + & \qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad 1_{two} \\ \hline = & 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1110_{two} \\ = & -2_{ten} \end{array}$$

Example (cont.)

- Negate 2_{ten}, and then check the result by negating -2_{ten}

Negating this number by inverting the bits and adding one,

$$\begin{array}{rl} & 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1101_{two} \\ + & \qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad 1_{two} \\ \hline = & 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1110_{two} \\ = & -2_{ten} \end{array}$$

Going the other direction,

$$1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1110_{two}$$

is first inverted and then incremented:

Example (cont.)

- Negate 2_{ten}, and then check the result by negating -2_{ten}

Negating this number by inverting the bits and adding one,

$$\begin{array}{rl} & 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1101_{two} \\ + & \qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad 1_{two} \\ \hline = & 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1110_{two} \\ = & -2_{ten} \end{array}$$

Going the other direction,

$$1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1110_{two}$$

is first inverted and then incremented:

$$\begin{array}{rl} & 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0001_{two} \\ + & \qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad 1_{two} \\ \hline = & 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0010_{two} \\ = & 2_{ten} \end{array}$$

Sign Extension

- To convert a binary number represented in *n* bits to a number represented with more than *n* bits
- For example, immediate field in load, store, branch, add, and set on less than instructions contains a two's complement 16-bit number
  - representing $-32{,}768_{ten}$ (-2^{15}) to $32{,}767_{ten}$ ($2^{15} - 1$)

Sign Extension

- To add the immediate field to a 32-bit register
  - Computer must convert that 16-bit number to its 32-bit equivalent
- Shortcut is to take MSB from the smaller quantity, sign bit, and replicate it to fill the new bits of the larger quantity
- Old non-sign bits are simply copied into the right portion of the new word
- Commonly called *sign extension*

Example

- Convert 16-bit binary versions of 2_{ten} and -2_{ten} to 32-bit binary numbers.

The 16-bit binary version of the number 2 is

$$0000\ 0000\ 0000\ 0010_{two} = 2_{ten}$$

It is converted to a 32-bit number by making 16 copies of the value in the most significant bit (0) and placing that in the left-hand half of the word. The right half gets the old value:

$$0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0010_{two} = 2_{ten}$$

Example (cont.)

Let's negate the 16-bit version of 2 using the earlier shortcut. Thus,

$$0000\ 0000\ 0000\ 0010_{two}$$

becomes

$$\begin{array}{l} 1111\ 1111\ 1111\ 1101_{two} \\ +\qquad\qquad\qquad\quad 1_{two} \\ \hline = \ 1111\ 1111\ 1111\ 1110_{two} \end{array}$$

Creating a 32-bit version of the negative number means copying the sign bit 16 times and placing it on the left:

$$1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1110_{two} = -2_{ten}$$

Notices

- Positive two's complement numbers have an infinite number of 0s on left
- Negative two's complement numbers have an infinite number of 1s on left
- Binary bit pattern representing a number hides leading bits to fit the width of HW
  - Sign extension simply restores some of them

Biased Notation

- A notation that represents
  - most negative value by $00 \ldots 000_{two}$
  - most positive value by $11 \ldots 11_{two}$
  - with 0 typically having the value $10 \ldots 00_{two}$,
- Thereby biasing number such that the number plus bias has a non-negative representation